🤖 AI Agent Context: Multi-Tenant Pre-Order Catering App

1. Project Overview

Build a Multi-Tenant Pre-Order Catering Mobile Application.

- **Target Audience:** Customers (ordering food via calendar) and Tenants (catering vendors managing daily quotas).
- **Environment:** Localhost backend for university project demonstration.
- **Tech Stack:**
  - **Frontend:** React Native (Expo), NativeWind (Tailwind), Zustand (State Management).
  - **Backend:** Node.js, Express.js, REST API.
  - **Database:** SQLite locally using Prisma ORM.
  - **Authentication:** Firebase Authentication (Cloud) synced with local SQLite.
  - **Payment:** Internal Dummy Payment Simulator (No external API).

2. Core Architecture Rules

- **Multi-Tenant Isolation:** All operational queries for `Tenant` users MUST include a `where: { tenantId: req.user.tenantId }` clause to prevent data leakage.
- **Auth Sync Strategy:** Firebase handles the login/register. Backend stores the `firebaseUid` in the local `User` table to attach `role` and `tenantId` context.
- **Pre-Order Logic:** Menus are attached to specific dates (`availableAt`). Orders are validated against `maxQuota` per menu per day.

3. Prisma Database Schema

Use the following schema exactly to initialize the SQLite database:

```
datasource db {
  provider = "sqlite"
  url      = "file:./dev.db"
```

```
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id          String   @id @default(uuid())
  firebaseUid String   @unique
  email       String   @unique
  name        String
  role        String   // ENUM: "CUSTOMER", "TENANT", "SUPER_ADMIN"
  tenantId    String?
  tenant      Tenant?  @relation(fields: [tenantId], references: [id])
  orders      Order[]
  createdAt   DateTime @default(now())
}

model Tenant {
  id          String   @id @default(uuid())
  name        String
  description String?
  address     String?
  users       User[]
  menus       Menu[]
  orders      Order[]
  createdAt   DateTime @default(now())
}

model Menu {
  id          String   @id @default(uuid())
  tenantId    String
  tenant      Tenant   @relation(fields: [tenantId], references: [id])
  name        String
  description String?
  price       Float
  maxQuota    Int      // Maximum portions available for this date
  availableAt String   // Format: "YYYY-MM-DD"
  orderItems  OrderItem[]
}

model Order {
  id            String      @id @default(uuid())
  customerId    String
  customer      User        @relation(fields: [customerId], references: [id])
  tenantId      String
  tenant        Tenant      @relation(fields: [tenantId], references: [id])
```

```
  totalAmount   Float
  paymentStatus String      // ENUM: "PENDING", "PAID", "FAILED"
  orderDate     DateTime    @default(now())
  orderItems    OrderItem[]
}

model OrderItem {
  id         String   @id @default(uuid())
  orderId    String
  order      Order    @relation(fields: [orderId], references: [id])
  menuId     String
  menu       Menu     @relation(fields: [menuId], references: [id])
  quantity   Int
  targetDate String   // Format: "YYYY-MM-DD"
}
```

4. Backend API Requirements (Express)

Create the following RESTful routes:

Auth & User Sync

- `POST /api/auth/sync`: Receives `{ firebaseUid, email, name, role }` from App. Creates user in SQLite if not exists.

Customer Endpoints

- `GET /api/tenants`: List all available caterings.
- `GET /api/tenants/:tenantId/menus?date=YYYY-MM-DD`: Get menus for a specific tenant on a specific date. Include remaining quota calculation (maxQuota - sum of paid order quantities).
- `POST /api/orders`: Create an order with `paymentStatus: "PENDING"`. Payload: `{ tenantId, items: [{ menuId, quantity, targetDate }] }`.

Dummy Payment Endpoint

- `PATCH /api/orders/:id/pay`: Receives `{ status: "PAID" | "FAILED" }`. Updates the order status in the database.

Tenant Endpoints (Must enforce `req.user.tenantId`)

- `GET /api/tenant/menus`: List all menus created by this tenant.

- `POST /api/tenant/menus`: Create a new daily menu.
- `GET /api/tenant/kitchen-rekap?date=YYYY-MM-DD`: Aggregate total quantities of each menu to be cooked for a specific date (only for orders where `paymentStatus` is "PAID").

5. Frontend UI/UX Requirements (React Native)

Navigators

- **AuthStack:** Login Screen, Register Screen (Handles Firebase Auth).
- **CustomerTab:** Home (Tenant List), Calendar (Menu selection per day), Cart/Checkout, Orders History.
- **TenantTab:** Dashboard (Kitchen Rekap), Menu Manager.

Dummy Payment Simulator Screen

- A standalone screen triggered after successful checkout.
- UI: Displays total amount and two large buttons:
  - **[Simulate Success]** -> Calls `PATCH /api/orders/:id/pay` with "PAID", navigates to Success Screen.
  - **[Simulate Failure]** -> Calls `PATCH /api/orders/:id/pay` with "FAILED", navigates to Error Screen.

6. Execution Plan for AI Agent

1. **Phase 1:** Initialize Express + Prisma + SQLite. Generate `schema.prisma`. Create `seed.ts` with 2 dummy tenants, 2 users, and 5 daily menus.
2. **Phase 2:** Build Express Middlewares (Firebase Token verifier). Implement the Endpoints described in Section 4.
3. **Phase 3:** Initialize Expo React Native project. Setup Zustand for state and NativeWind for UI.
4. **Phase 4:** Implement Firebase Client Auth in React Native and integrate with `/api/auth/sync`.
5. **Phase 5:** Build Customer flow (Home -> Calendar Menu -> Checkout -> Dummy Payment Screen).
6. **Phase 6:** Build Tenant flow (Kitchen Rekap view).